

Lab4实验报告

思考题

Thinking 4.1

Thinking 4.1 思考并回答下面的问题：

- 内核在保存现场的时候是如何避免破坏通用寄存器的？
 - 系统陷入内核调用后可以直接从当时的 `$a0-$a3` 参数寄存器中得到用户调用 `msyscall` 留下的信息吗？
 - 我们是怎么做到让 `sys` 开头的函数“认为”我们提供了和用户调用 `msyscall` 时同样的参数的？
 - 内核处理系统调用的过程对 `Trapframe` 做了哪些更改？这种修改对应的用户态的变化是什么？
- 内核在保护现场调用了 `include/stackframe.h` 中的 `SAVE_ALL` 宏，把通用寄存器保存到了栈帧里。
 - 可以。因为从陷入内核到调用 `syscall` 之间没有对相关寄存器进行任何修改。
 - 从 `msyscall` 到 `sys_*` 之间，栈指针未出现改变，指向同一个上下文，且 `msyscall` 的参数入栈顺序与 `sys_*` 函数传参顺序一致。
 - 将 `EPC` 值加4，使返回用户态时，返回 `syscall` 的下一条指令。

Thinking 4.2

Thinking 4.2 思考 `envid2env` 函数：为什么 `envid2env` 中需要判断 `e->env_id != envid` 的情况？如果没有这步判断会发生什么情况？

- `mkenvid` 为每一个进程申请一个唯一的 `id`，代码如下：

```
1 u_int mkenvid(struct Env *e) {
2     static u_int i = 0;
3     return ((++i) << (1 + LOG2NENV)) | (e - envs);
4 }
```

可以看到，所获 `id` 的低十位只与 `e - envs` 有关，而注意到宏 `ENVX`：本质就是取 `envid` 的低十位。

```
1 #define LOG2NENV 10
2 #define NENV (1 << LOG2NENV)
3 #define ENVX(envid) ((envid) & (NENV - 1))
```

由以上代码可以得知，对于同一个 `Env` 结构体，其所对应的得到的 `id` 的低十位是一定的，故通过 `ENVX` 获取的偏移量也是一定的。

虽然进程是可以申请和销毁的，但进程结构体 `Env` 缺失复用的。若用一个已经销毁的进程的 `id` 取调用 `envid2env` 函数，通过 `ENVX` 取 `envs` 数组后，可能会得到一个未被销毁的进程结构体，但这是不允许的，故需要确保 `id` 一致从而避免这种情况发生。

Thinking 4.3

Thinking 4.3 思考下面的问题，并对这个问题谈谈你的理解：请回顾 `kern/env.c` 文件中 `mkenvid()` 函数的实现，该函数不会返回 0，请结合系统调用和 IPC 部分的实现与 `envid2env()` 函数的行为进行解释。

- 代码如下：

```
1 u_int mkenvid(struct Env *e) {
2     static u_int i = 0;
3     return ((++i) << (1 + LOG2NENV)) | (e - envs);
4 }
```

这个函数通过拼接一个非0的整数 `i` 与待分配进程块偏移量生成新的 `envid`，这意味着它生成的 `envid` 永远不为0。

- IPC需要在两个不同进程间传递信息。如果 `envid2env()` 返回当前进程，IPC相关系统调用函数就会失效（自己向自己传送信息）；另一方面，取消 `envid2env()` 的返回当前进程功能的话，不便于部分系统调用的实现。因此需要保留返回自己的选项（以0为参数传入）。

Thinking 4.4

Thinking 4.4 关于 `fork` 函数的两个返回值，下面说法正确的是：

- A、`fork` 在父进程中被调用两次，产生两个返回值
- B、`fork` 在两个进程中分别被调用一次，产生两个不同的返回值
- C、`fork` 只在父进程中被调用了一次，在两个进程中各产生一个返回值
- D、`fork` 只在子进程中被调用了一次，在两个进程中各产生一个返回值

- 正确的是C

Thinking 4.5

Thinking 4.5 我们并不应该对所有的用户空间页都使用 `duppage` 进行映射。那么究竟哪些用户空间页应该映射，哪些不应该呢？请结合 `kern/env.c` 中 `env_init` 函数进行的页面映射、`include/mmu.h` 里的内存布局图以及本章的后续描述进行思考。

- 映射 `0 ~ USTACKTOP` 之间的空间。`USTACKTOP ~ UXSTACKTOP` 的空间为异常栈，是保存异常处理信息的位置，无需映射；`UXSTACKTOP` 以上空间不在用户空间内，无需映射。

Thinking 4.6

Thinking 4.6 在遍历地址空间存取页表项时你需要使用到 `vpt` 和 `vpd` 这两个指针，请参考 `user/include/lib.h` 中的相关定义，思考并回答这几个问题：

- `vpt` 和 `vpd` 的作用是什么？怎样使用它们？
- 从实现的角度谈一下为什么进程能够通过这种方式来存取自身的页表？
- 它们是如何体现自映射设计的？
- 进程能够通过这种方式来修改自己的页表项吗？
- `vpt` 是页表基地址，`vpd` 是页目录基地址，用于查询虚拟地址所对应的页目录项或页表项。使用 `vpd[页目录项偏移值]` 或 `vpt[页表项偏移值]`，即类似于数组的索引形式。
- 进程共用一个页表，不同进程的页面通过不同的 `asid` 来区分，故可以直接通过 `vpt` 读取自身页表。`vpt` 的基地址在虚拟地址空间是固定的，为 `UVPT`。
- 宏定义代码如下

```
1 #define vpt ((const volatile Pte *)UVPT)
2 #define vpd ((const volatile Pde *) (UVPT + (PDX(UVPT) << PGSHIFT)))
```

自映射设计体现在 `vpt` 和 `vpd` 直接将页表和页目录映射到进程的虚拟地址空间中。通过这种方式，进程可以像访问普通内存一样访问自己的页表和页目录。这种设计使得进程无需通过特殊的系统调用或硬件操作来读取或修改自己的页表项，从而简化了内存管理的实现。自映射允许进程通过计算偏移直接访问页表项，这是通过将页表映射到虚拟地址空间的一个固定位置来实现的，如 `UVPT` 所示。

- 不可以。修改页表项只能在内核态下进行，必须调用系统调用，陷入内核后进行操作。

Thinking 4.7

Thinking 4.7 在 `do_tlb_mod` 函数中，你可能注意到了一个向异常处理栈复制 `Trapframe` 运行现场的过程，请思考并回答这几个问题：

- 这里实现了一个支持类似于“异常重入”的机制，而在什么时候会出现这种“异常重入”？
- 内核为什么需要将异常的现场 `Trapframe` 复制到用户空间？
- 当用户程序在处理一个异常时，又发生了另一个异常。例如，用户程序在处理一个TLB缺失异常时，其异常处理代码又触发了另一个TLB缺失，又或者用户程序处理异常时触发时钟中断。
- 根据MOS的微内核设计理念，允许用户程序在用户空间处理某些异常，如TLB缺失。这样做可以减少内核态和用户态之间的切换开销，提高效率。故需要把现场复制到用户空间供用户空间处理异常。

Thinking 4.8

Thinking 4.8 在用户态处理页写入异常，相比于在内核态处理有什么优势？

1. 减少开销：

- 当异常在用户态处理时，不需要进行完整的内核态和用户态之间的上下文切换。这种切换涉及到保存和恢复寄存器状态、切换地址空间等，这些都是耗时的操作。在用户态直接处理可以省去这些开销，从而提高效率。

2. 提高速度：

- 用户态处理异常可以更快地响应异常情况，因为不需要等待内核介入。这对于实时系统或者对响应时间有严格要求的应用来说是非常重要的。

3. 降低内核复杂性：

- 将异常处理放在用户态可以减少内核的复杂性。内核代码通常需要处理各种异常情况，而且需要保证高度的安全性和稳定性。将部分异常处理逻辑下放到用户态可以简化内核的设计和实现。

4. 增强灵活性：

- 用户态程序可以根据自己的需要定制异常处理逻辑。例如，应用程序可能想要实现特定的内存管理策略，或者在异常发生时执行特定的恢复操作。在用户态处理异常提供了这种灵活性。

5. 更好的错误隔离：

- 在用户态处理异常可以更好地隔离错误。如果异常处理代码出现错误，它只会影响当前进程，而不会影响整个内核。这有助于提高系统的稳定性和可靠性。

6. 优化资源利用：

- 用户态程序可能对自身的内存使用模式有更深入的了解，因此能够更有效地处理页写入异常。例如，程序可能知道哪些页面是临时使用的，可以立即释放，从而优化内存资源的使用。

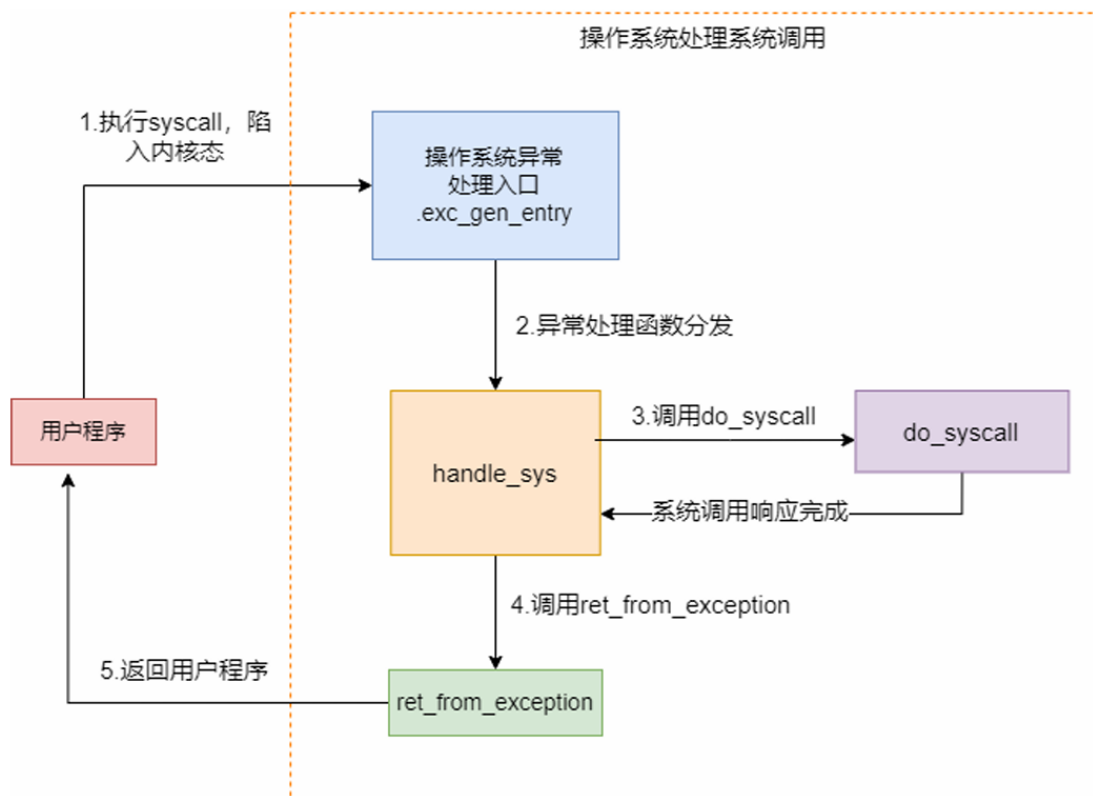
Thinking 4.9

Thinking 4.9 请思考并回答以下几个问题：

- 为什么需要将 `syscall_set_tlb_mod_entry` 的调用放置在 `syscall_exofork` 之前？
- 如果放置在写时复制保护机制完成之后会有怎样的效果？
- `syscall_exofork` 后子进程被创建，父子进程都需要执行 `syscall_set_tlb_mod_entry`，父进程已经为子进程写入了异常函数入口位置，所以可以更改位置。否则，子进程将不能正确处理异常。
- 父进程运行时会修改栈。在栈空间的页面标记为写时复制之后，父进程继续运行并修改栈，就会触发 `TLB Mod` 异常。所以在写时复制保护机制完成之前就需要 `syscall_set_tlb_mod_entry`。

难点与体会

其实本次实验最重要的是理解系统调用处理流程



完整流程涉及非常多的函数与理论知识，读起来很吃力，尤其是涉及COW的部分，对于内存管理与复制的部分非常生疏，只能参考往届学长博客和课程组提供的代码才能勉强理解一些。

此外，对于tlb相关内容我仍然不太掌握，需要多加努力。